RAPPORT DE TRAVAUX PRATIQUES

UNIVERSITÉ SULTAN MOULAY SLIMANE
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES
BÉNI MELLAL

TP3 – Architecture distribuée

Réalisé par : EL MELIANI Ammar
Encadrant : Pr. EL BAGHAZAOUI

Filière : **IACS-S2**
Année universitaire : 2025/2026

Lien GITHUB :

https://github.com/EL-Star456/TP_Rapport_Log.git

TP 3 – Architecture distribuée

1. Objectifs du TP :

Le TP3 simule un vrai système e-commerce distribué avec 6 composants qui communiquent ensemble :

Postman

Le client qui envoie les requetes HTTP — comme toi qui passes une commande sur un site.

Nginx :80

Le load balancer — comme un receptionniste qui distribue les appels entre plusieurs employes.

Node.js x2

2 serveurs applicatifs (port 3001 et 3002) — la logique metier, traitement des commandes

MongoDB

Base de donnees NoSQL — stocke toutes les commandes de facon persistante

Redis

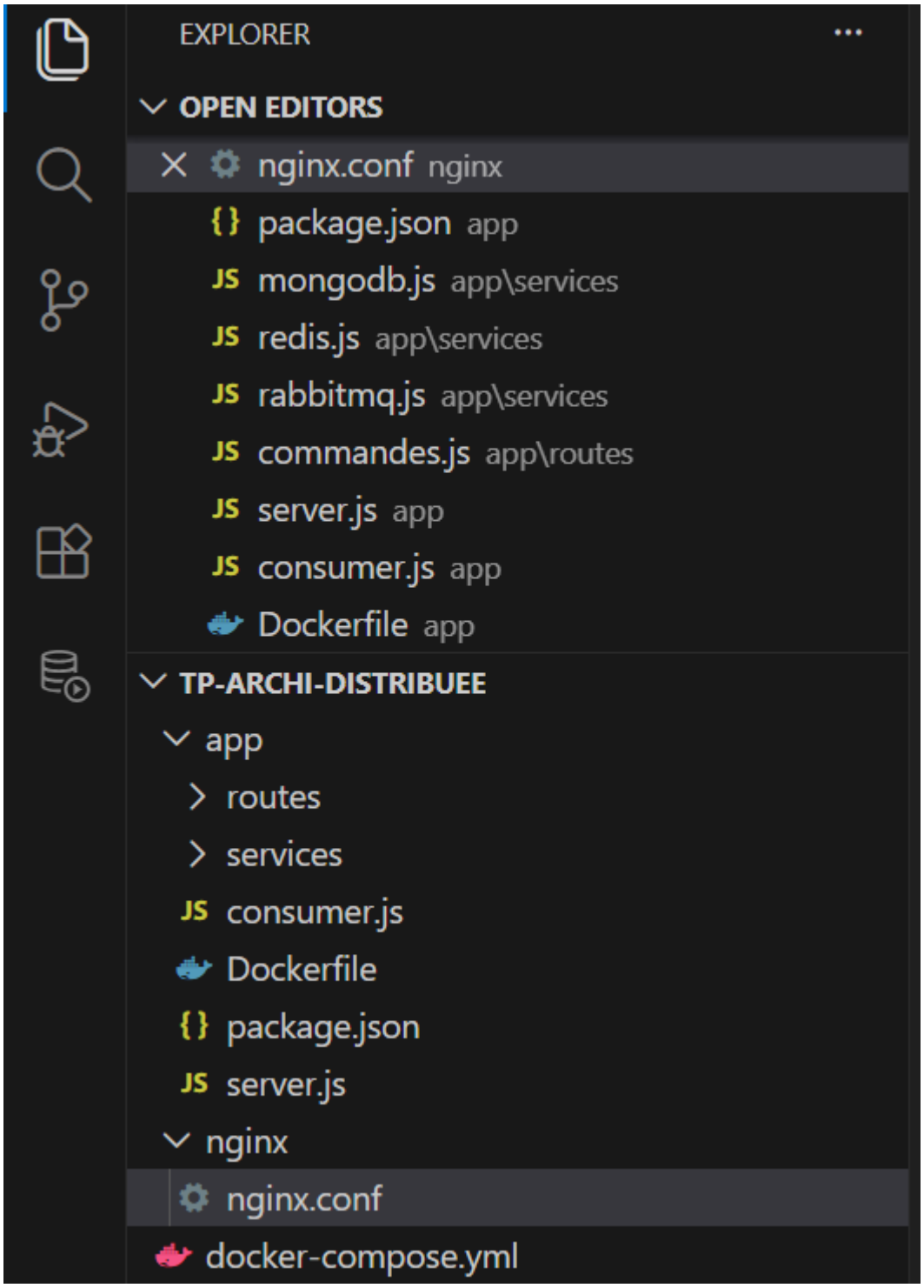
Cache memoire — evite de relire MongoDB a chaque requete GET (ultra rapide)

RabbitMQ

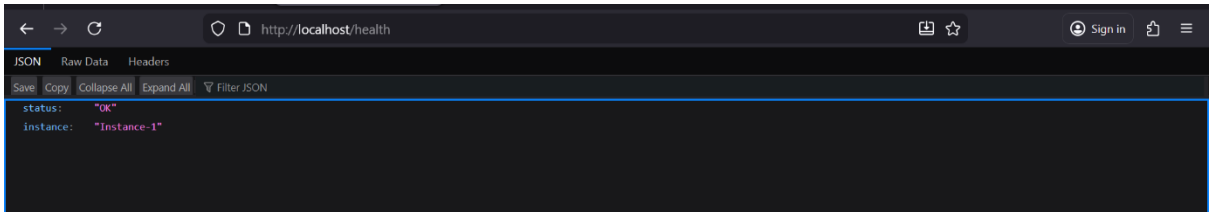
Bus de messages — envoie une notification apres chaque commande (comme un email de confirmation).

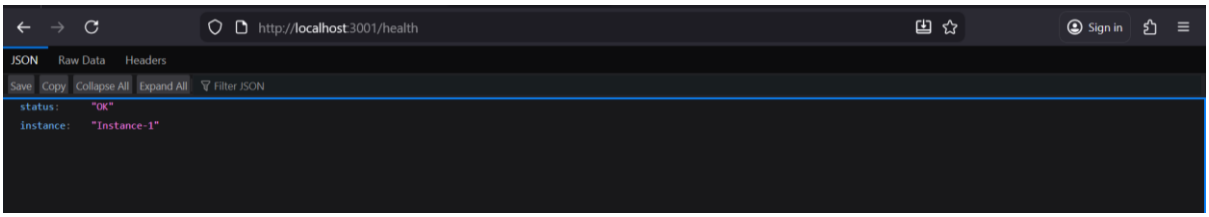
```
[+] Running 8/8
 ✓ Network tp-archi-distribuee_default      Created                    0.2s
 ✓ Volume "tp-archi-distribuee_mongo_data"  Created                    0.0s
 ✓ Container rabbitmq                       Created                    0.3s
 ✓ Container redis                          Created                    0.3s
 ✓ Container mongodb                        Created                    0.4s
 ✓ Container app2                           Created                    0.3s
 ✓ Container app1                           Created                    0.3s
 ✓ Container nginx                          Created                    0.1s
```

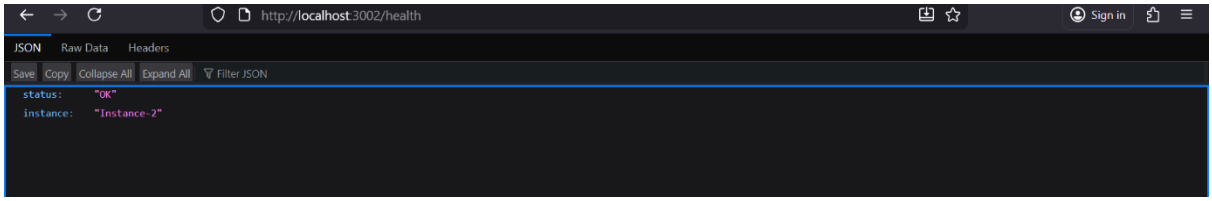
2. Architecture :



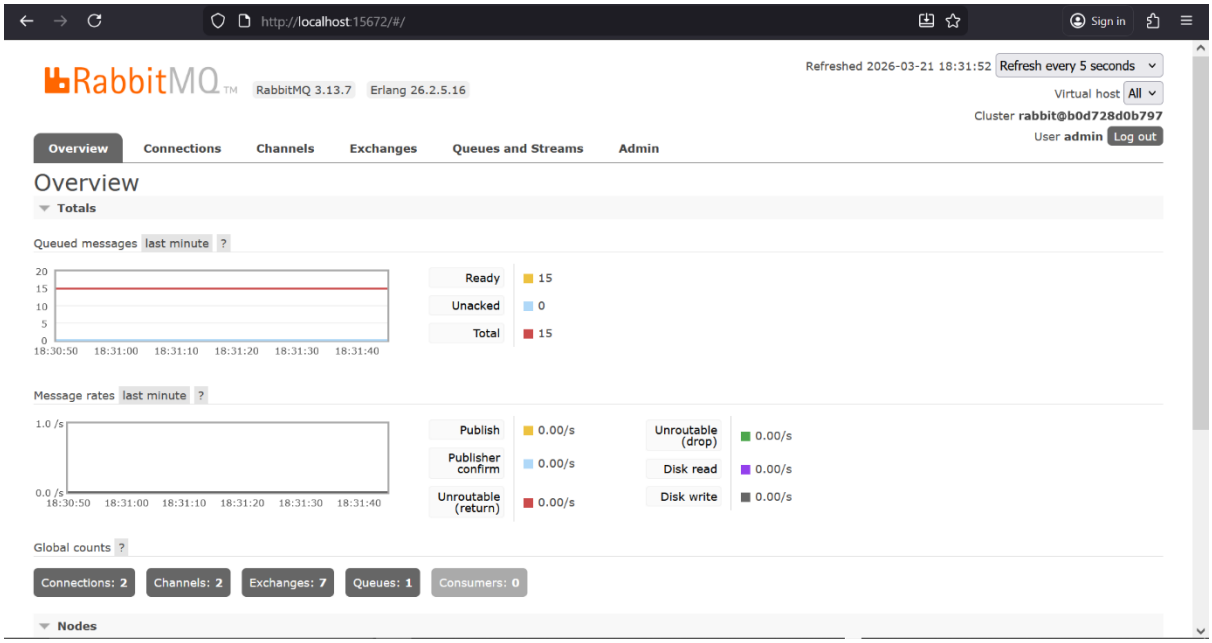
3. URLs :


http://localhost/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-1"


http://localhost:3001/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-1"


http://localhost:3002/health
JSON
Raw Data
Headers
Save
Copy
Collapse All
Expand All
Filter JSON
status: "OK"
instance: "Instance-2"

http://localhost:15672
RabbitMQ
Username:
Password:
Login


http://localhost:15672/#/
RabbitMQ
RabbitMQ 3.13.7
Erlang 26.2.5.16
Refreshed 2026-03-21 18:31:52
Refresh every 5 seconds
Virtual host All
Cluster rabbit@b0d728d0b797
User admin
Log out
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Overview
Totals
Queued messages last minute
Ready 15
Unacked 0
Total 15
Message rates last minute
Publish 0.00/s
Publisher confirm 0.00/s
Unroutable (return) 0.00/s
Unroutable (drop) 0.00/s
Disk read 0.00/s
Disk write 0.00/s
Global counts
Connections: 2
Channels: 2
Exchanges: 7
Queues: 1
Consumers: 0
Nodes

4.Test avec postman :

POST http://localhost/commandes Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2 "produit" : "Laptop Dell XPS",
3 "quantite" : 2,
4 "client" : "Ahmed Benali"
5 }
```

Body 201 Created 27 ms 472 B

{ } JSON Preview Visualize

```
1 {
2     "message": "Commande créée",
3     "commande": {
4         "produit": "Laptop Dell XPS",
5         "quantite": 2,
6         "client": "Ahmed Benali",
7         "statut": "en_attente",
8         "_id": "69bedda74b37b8e8e952ca46",
9         "date": "2026-03-21T18:04:23.723Z",
```

http://localhost/commandes Save Share

GET http://localhost/commandes Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2 "produit" : "Laptop Dell XPS",
3 "quantite" : 2,
4 "client" : "Ahmed Benali"
5 }
```

Body 200 OK 58 ms 764 B

{ } JSON Preview Visualize

```
1 {
2     "source": "MongoDB",
3     "traitee_par": "Instance-2",
4     "commandes": [
5         {
6             "_id": "69bedda74b37b8e8e952ca46",
7             "produit": "Laptop Dell XPS",
8             "quantite": 2,
9             "client": "Ahmed Benali",
10            "statut": "en_attente",
11            "date": "2026-03-21T18:04:23.723Z",
12            "__v": 0
13        },
14        {
15            "_id": "69bedd523a770e92ee291a47",
16            "produit": "Laptop",
17            "quantite": 1
```

Body 200 OK 18 ms 2.58 KB

{} JSON Preview Visualize


```
{
    "source": "⚡ CACHE Redis",
    "traitee_par": "Instance-2",
    "commandes": [
        {
            "_id": "69beded33a770e92ee291a54",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-21T18:09:23.169Z",
            "__v": 0
        },
        {
            "_id": "69bedeca4b37b8e8e952ca52",
```

```
            "_id": "69bedec53a770e92ee291a52",
            "produit": "Laptop Dell XPS",
            "quantite": 2,
            "client": "Ahmed Benali",
            "statut": "en_attente",
            "date": "2026-03-21T18:09:09.600Z",
            "__v": 0
        },
```


5. Questions :

Q1/Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

Le composant qui reçoit les requêtes en premier est le Load Balancer (ici Nginx). Il agit comme une porte d'entrée unique pour le système. L'algorithme utilisé est le Round-robin. Il distribue les requêtes de manière équitable et séquentielle : la première requête va vers `app1`, la deuxième vers `app2`, la troisième revient à `app1`, et ainsi de suite.

Q2/Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Si `app1` tombe en panne, Nginx continue de fonctionner normalement. Pourquoi ? Parce que Nginx est un composant indépendant des serveurs applicatifs. Sa configuration lui permet de détecter qu'une instance ne répond plus (mécanisme de *Health Check*). Il retire alors temporairement `app1` de sa liste et redirige tout le trafic vers les instances saines (comme `app2`). C'est le principe de la Haute Disponibilité.

Q3/Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET

- **1er GET (Source MongoDB) : La requête est plus lente car l'application doit lire les données sur le disque dur via MongoDB. Une fois l'information récupérée, elle est stockée dans Redis.**

- **2ème GET (Source Redis) : La réponse est quasi instantanée. L'application récupère la donnée directement depuis la mémoire vive (RAM) de Redis sans solliciter la base de données principale.**

Q4/Pourquoi invalide-t-on le cache Redis après un POST /commande

On invalide (supprime) le cache pour garantir la cohérence des données. Si un utilisateur crée une nouvelle commande via un `POST`, les données stockées dans MongoDB changent. Si on ne supprimait pas le cache Redis, le prochain `GET` renverrait l'ancienne liste des commandes (sans la nouvelle) car Redis ne saurait pas que la base de données a été mise à jour. En supprimant le cache, on force le système à refaire un `GET` propre sur MongoDB lors de la prochaine demande.

Q5/En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ améliore la résilience grâce au découplage asynchrone :

1. **Isolation des pannes : Si le service de notification (consommateur) tombe en panne, le service de commande (producteur) peut continuer à fonctionner. Les messages ne sont pas perdus, ils sont stockés en sécurité dans la file d'attente (Queue).**
2. **Reprise après sinistre : Dès que le service de notification redémarre, il consomme tous les messages accumulés dans RabbitMQ. Le système "rattrape" son retard sans perte de données**.

Q6/Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dansnginx.conf ?

Pour ajouter une instance (par exemple `app3` sur le port `3003`), il faudrait modifier le bloc `upstream` dans le fichier `nginx.conf`.

Fin